

Python Practice Questions - 2025

Python - Senior Developer Level

1. Python Datatypes & Built-in Functions

Primitive Datatypes

Q1.1 (Coding): You're working on a patient billing system. Write a function that calculates the total cost for a patient's hospital stay, handling different data types appropriately.

```
def calculate_patient_bill(room_charge_per_day: float, days_stayed: int,
                           has_insurance: bool, patient_name: str) -> dict:
    """
    Calculate total bill for a patient
    - Apply 20% discount if patient has insurance
    - Return a dictionary with patient details and final amount
    """
    # Your implementation here
    pass
```

Expected Output: Handle type conversions, boolean logic, and string formatting.

```
def calculate_patient_bill(room_charge_per_day: float, days_stayed: int,
                           has_insurance: bool, patient_name: str) -> dict:

    Amount= room_charge_per_day* days_stayed;

    if has_insurance:

        Amount= Amount*0.80;

    return { "patient_name" : patient_name , "Amount": Amount }
```

```
print(calculate_patient_bill(100, 5, True, "John"))
```

```
print(calculate_patient_bill(100, 5, False, "John"))
```

Q1.2 (Reasoning): In a healthcare database, patient IDs are sometimes stored as integers (12345) and sometimes as strings ("P12345"). Explain the memory allocation differences between these two approaches and when you would use each in a hospital management system.

Int:

ABC_1

List Operations

Q1.3 (Coding): You're managing a patient's medication schedule. Create a system to track daily medications:

```
def manage_medication_schedule(patient_medications: list) -> dict:
    """
    Given a list of medications with their schedules:
    [('Aspirin', '8:00'), ('Insulin', '8:00'), ('Aspirin', '20:00')]

    Return a dictionary grouped by time:
    {'8:00': ['Aspirin', 'Insulin'], '20:00': ['Aspirin']}
```

Also implement methods to:

1. Add new medication
2. Remove medication at specific time
3. Get all medications for a specific time

"""

pass

```
def manage_medication_schedule(patient_medications: list) -> dict:
```

```
    print(patient_medications)
```

```
    result = {}
```

```
    for medication in patient_medications:
```

```
        time = medication[1]
```

```
        if time not in result:
```

```
            result[time] = []
```

```
            result[time].append(medication[0])
```

```
    print(result["8:00"])
```

```
    return result
```

```
result = manage_medication_schedule([

("Aspirin", "8:00"),

("Insulin", "8:00"),

("Aspirin", "20:00"),

])


print(result)
```

Q1.4 (Reasoning): What is the underlying data structure of Python lists? How does this affect performance when you're inserting patient records at the beginning vs. end of a large patient queue? Compare with Java's ArrayList.

Tuple Operations

Q1.5 (Coding): Design a system to store immutable patient vital signs readings:

```
def process_vital_signs(readings: list) -> tuple:
    """
    Process vital signs readings and return statistics
    Input: [(120, 80, 98.6, '2024-01-15'), (115, 75, 99.1, '2024-01-16')]
    Output: Tuple of (avg_systolic, avg_diastolic, max_temp, date_range)
    """
    pass
```

```
def process_vital_signs(readings: list) -> tuple:
```

```
    """
```

```
    Process vital signs readings and return statistics
```

```
    Input: [(120, 80, 98.6, '2024-01-15'), (115, 75, 99.1, '2024-01-16')]
```

```
    Output: Tuple of (avg_systolic, avg_diastolic, max_temp, date_range)
```

```
    """
```

```
    systolic_list = []
```

```
    diastolic_list = []
```

```
    temp_list = []
```

```
    date_list = []
```

```
    for reading in readings:
```

```
        systolic, diastolic, temp, date = reading
```

```
        systolic_list.append(systolic)
```

```
        diastolic_list.append(diastolic)
```

```
        temp_list.append(temp)
```

```
        date_list.append(date)
```

```
    avg_systolic = sum(systolic_list) / len(systolic_list)
```

```
    avg_diastolic = sum(diastolic_list) / len(diastolic_list)
```

```
    max_temp = max(temp_list)
```

```
    date_range = (min(date_list), max(date_list))
```

```
return avg_systolic, avg_diastolic, max_temp, date_range
```

```
result = process_vital_signs([(120, 80, 98.6, '2024-01-15'), (115, 75, 99.1, '2024-01-16')])
```

```
print(result)
```

```
def process_vital_signs(readings: list) -> tuple:
```

```
    """
```

```
    Process vital signs readings and return statistics
```

```
    Input: [(120, 80, 98.6, '2024-01-15'), (115, 75, 99.1, '2024-01-16')]
```

```
    Output: Tuple of (avg_systolic, avg_diastolic, max_temp, date_range)
```

```
    """
```

```
    systolic_list, diastolic_list, temp_list, date_list = zip(*readings)
```

```
    avg_systolic = sum(systolic_list) / len(systolic_list)
```

```
    avg_diastolic = sum(diastolic_list) / len(diastolic_list)
```

```
    max_temp = max(temp_list)
```

```
    date_range = (min(date_list), max(date_list))
```

```
    return avg_systolic, avg_diastolic, max_temp, date_range
```

```
result = process_vital_signs([(120, 80, 98.6, '2024-01-15'), (115, 75, 99.1, '2024-01-16')])
```

```
print(result)
```

Q1.6 (Reasoning): Why does `temp_reading = (98.6)` create a float instead of a tuple, but `temp_reading = (98.6,)` creates a tuple? How does this relate to storing single vital sign measurements in healthcare applications?

```
def average_temps(readings):
```

```
    # API expects a sequence: (98.6,) or [98.6, 99.1]
```

```
    return sum(readings) / len(readings)
```

```
    # INPUT — scalar float fails
```

```
    try:
```

```
        print(average_temps(98.6))
```

```
    except TypeError as error:
```

```
        print(type(error).__name__)
```

```
    # OUTPUT
```

```
    TypeError
```

```
    # INPUT — one-item tuple succeeds
```

```
    print(average_temps((98.6,)))
```

```
    # OUTPUT
```

98.6

```
def average_temps(readings: tuple) -> float:

    # API expects a sequence: (98.6,) or [98.6, 99.1]

    return sum(readings) / len(readings)


# INPUT — type hints do not prevent this run-time error

try:

    print(average_temps(98.6))

except TypeError as error:

    print(type(error).__name__)


# OUTPUT

TypeError


# INPUT — tuples still work

print(average_temps((98.6,)))

print(average_temps((98.6, 99.1)))


# OUTPUT
```


98.6

98.85

- mypy/pyright can **warn early** that 98.6 is not a tuple---Type Error.

Q1.7 (Reasoning): Compare the memory footprint and access speed of tuples vs lists when storing thousands of patient coordinate data (x, y, z) for medical imaging. Why would you choose one over the other?

Memory=tuple less memory, list more memory. (list- append/insert at begin costly, in that case use deque.)

Speed=tuple and list also same speed. $O(1)$. Tuple little bit faster.

Growing items I will use list. Fixed items I will use tuple.

Dictionary Operations

Q1.8 (Coding): Create a patient record management system using dictionaries:

```
def patient_record_system():  
    """  
    Create a system that manages patient records with nested dictionaries:  
    {  
        'patient_id': {  
            'personal': {'name': 'John', 'age': 45, 'gender': 'M'},  
            'medical': {'allergies': ['penicillin'], 'blood_type': 'O+'},  
            'visits': [{'date': '2024-01-15', 'diagnosis': 'Hypertension'}]  
        }  
    }  
    """  
  
    Implement:  
    1. Add new patient  
    2. Update patient information  
    3. Search patients by blood type  
    4. Get patient visit history
```

```
"""  
pass
```

Step 1: Think of a filing cabinet: each patient gets one folder (by id) with personal info, medical info, and visits. We add folders, update them, and search by bloo...

class PatientRecords:

def __init__(self):

Step 2: keep records in a dictionary keyed by patient id

self.records: dict[str, dict] = {}

def add(self, patient_id: str, personal: dict, medical: dict) -> None:

Step 3: reject duplicates; store personal, medical, empty visits

if patient_id in self.records:

raise ValueError("patient already exists")

self.records[patient_id] = {

"personal": dict(personal),

"medical": dict(medical),

"visits": [],

}

```
def update(self, patient_id: str, **changes) -> None:
```

```
# Step 4: merge new fields into the existing record
```

```
self.records[patient_id].update(changes)
```

```
def by_blood_type(self, blood_type: str) -> list[str]:
```

```
# Step 5: collect ids whose medical blood type matches
```

```
return [
```

```
    pid
```

```
    for pid, r in self.records.items()
```

```
        if r["medical"].get("blood_type", "").upper() == blood_type.upper()
```

```
]
```

```
def visits(self, patient_id: str) -> list[dict]:
```

```
# Step 6: return a copy of that patient's visits
```

```
return list(self.records[patient_id]["visits"])
```

```
if __name__ == "__main__":
```

```
    db = PatientRecords()
```

```
db.add("P1", {"name": "John", "age": 45}, {"blood_type": "O+", "allergies":  
["penicillin"]})
```

```
print(db.by_blood_type("o+"))
```

Q1.9 (Reasoning): What types of objects can be used as dictionary keys in Python? Why can't you use a list of patient symptoms as a dictionary key, but you can use a tuple of symptoms? Explain with healthcare examples.

Int, str, tuple

Python uses a **hash** to find the locker quickly. Keys must be **hashable**: they have a stable hash and clear equality.

Q1.10 (Reasoning): What's the difference between `patient_record.get('blood_type', 'Unknown')` and `patient_record['blood_type']` when accessing potentially missing medical data? When would each approach be appropriate in a healthcare system?

patient[key] - if key present it will give value if not, it will return Type error. present, so use for **Required fields**

patient.get(key): if key present it will give value if not, it will return None. so use for **Optional fields**,

Set Operations

Q1.11 (Coding): Implement a drug interaction checker using sets:

```
def check_drug_interactions(current_medications: set, new_medication: str,  
                             interaction_db: dict) -> dict:  
    """  
    Check for drug interactions before prescribing new medication
```

```

interaction_db = {
    'aspirin': {'warfarin', 'heparin'},
    'warfarin': {'aspirin', 'vitamin_k'},
    ...
}

Return: {
    'safe': bool,
    'interactions': set of conflicting drugs,
    'recommendations': list
}
"""
pass

```

Q1.12 (Reasoning): How does Python's set implement hashing internally? Why is this crucial for fast lookups in medical databases containing millions of patient records?

2. Typing

Q2.1 (Coding): Create a strongly-typed patient data validation system:

```

from typing import List, Dict, Optional, Union, Tuple
from dataclasses import dataclass
from enum import Enum

class BloodType(Enum):
    A_POS = "A+"
    A_NEG = "A-"
    B_POS = "B+"
    B_NEG = "B-"
    AB_POS = "AB+"
    AB_NEG = "AB-"
    O_POS = "O+"
    O_NEG = "O-"

@dataclass
class VitalSigns:
    systolic: int

```

```

    diastolic: int
    temperature: float
    heart_rate: int
    timestamp: str

def validate_patient_data(patient_id: str,
                           age: int,
                           blood_type: BloodType,
                           vitals: List[VitalSigns],
                           emergency_contact: Optional[Dict[str, str]] = None) -> bool:
    """
    Validate patient data with proper type hints
    """
    pass

```

Q2.2 (Reasoning): How do Python's type hints differ from Java's static typing? What are the benefits of using type hints in a large healthcare codebase with multiple developers?

3. Operators

Arithmetic Operators

Q3.1 (Coding): Calculate BMI and medication dosage using various arithmetic operators:

```

def medical_calculations(weight_kg: float, height_m: float,
                          base_dosage_mg: float, patient_age: int) -> dict:
    """
    Calculate:
    1. BMI using weight and height
    2. Adjusted medication dosage (reduce by 10% for patients over 65)
    3. Days supply if total medication is 1000mg and dosage is calculated above
    4. Whether patient is in healthy BMI range (18.5-24.9)
    """
    pass

```

Comparison and Logical Operators

Q3.2 (Coding): Create a patient triage system using comparison operators:

```
def triage_patient(temperature: float, blood_pressure: tuple,
                  heart_rate: int, is_conscious: bool,
                  pain_level: int) -> str:
    """
    Determine patient priority based on vital signs:
    - Critical: Temp > 104°F OR BP > (180, 120) OR Heart rate > 120 OR unconscious
    - Urgent: Temp > 101°F OR BP > (140, 90) OR Pain > 7
    - Standard: All other cases

    Use appropriate comparison and logical operators
    """
    pass
```

Bitwise Operators

Q3.3 (Coding): Implement a patient permission system using bitwise operators:

```
class PatientPermissions:
    """
    Use bitwise operators to manage patient data access permissions:
    READ = 1 (001)
    WRITE = 2 (010)
    DELETE = 4 (100)
    """
    READ = 1
    WRITE = 2
    DELETE = 4

    def __init__(self, permissions: int = 0):
        self.permissions = permissions

    def add_permission(self, permission: int):
        """Add permission using bitwise OR"""
        pass

    def remove_permission(self, permission: int):
        """Remove permission using bitwise operations"""
        pass
```

```
def has_permission(self, permission: int) -> bool:
    """Check if permission exists using bitwise AND"""
    pass
```

Membership and Identity Operators

Q3.4 (Coding): Implement patient eligibility checker:

```
def check_patient_eligibility(patient_age: int, symptoms: list,
                               insurance_types: list, medical_history: list) -> dict:
    """
    Use 'in' and 'not in' operators to check:
    1. If patient has any critical symptoms
    2. If patient's insurance is accepted
    3. If patient has conflicting medical history

    Use 'is' and 'is not' for None checks on optional data
    """
    critical_symptoms = ['chest_pain', 'difficulty_breathing', 'severe_bleeding']
    accepted_insurance = ['medicare', 'aetna', 'bcbs', 'cigna']

    pass
```

Walrus Operator

Q3.5 (Coding): Use walrus operator for efficient patient data processing:

```
def process_patient_vitals(patients_data: list) -> list:
    """
    Use walrus operator to efficiently process and filter patient data:
    - Calculate BMI and filter patients with BMI > 30
    - Process temperature readings and flag fever cases
    - Extract medication counts and identify patients with > 5 medications
    """
    results = []

    for patient in patients_data:
        # Use walrus operator for BMI calculation and filtering
```



```
# Use walrus operator for temperature processing
# Use walrus operator for medication counting
pass

return results
```

4. Conditional Flow Control

Q4.1 (Coding): Create a comprehensive patient diagnosis workflow:

```
def diagnose_patient(symptoms: list, vital_signs: dict,
                    medical_history: list, lab_results: dict) -> dict:
    """
    Implement a diagnosis decision tree using if-else, nested conditions:

    Priority flow:
    1. Check for emergency conditions first
    2. Evaluate chronic condition indicators
    3. Consider common ailments
    4. Default to further testing required

    Use nested if-else appropriately for complex medical decision making
    """
    pass

def patient_monitoring_loop(patient_id: str, monitoring_duration_hours: int):
    """
    Use while loop to simulate patient vital signs monitoring
    - Check vitals every 30 minutes
    - Alert if vitals go outside normal ranges
    - Break if patient is stable for 2 consecutive hours
    """
    pass

def daily_medication_schedule(patient_medications: dict):
    """
    Use for loop to generate daily medication reminders
    - Iterate through medication times
```

```
- Handle special cases (meals, sleep time)
- Generate patient-friendly schedule
"""
pass
```

5. Comprehensions

Q5.1 (Coding): Implement efficient data processing using comprehensions:

```
def medical_data_processing(patient_records: list) -> dict:
    """
    Use different comprehensions to process medical data:

    1. List comprehension: Extract all patients with diabetes
    2. Dictionary comprehension: Map patient IDs to their latest vital signs
    3. Set comprehension: Get unique medications across all patients
    4. Generator expression: Memory-efficient processing of large patient datasets
    """

    # List comprehension for diabetic patients
    diabetic_patients = # Your code here

    # Dictionary comprehension for latest vitals
    latest_vitals = # Your code here

    # Set comprehension for unique medications
    unique_medications = # Your code here

    # Generator for memory-efficient processing
    def high_risk_patients():
        # Generator expression here
        pass

    return {
        'diabetic_patients': diabetic_patients,
        'latest_vitals': latest_vitals,
        'unique_medications': unique_medications,
        'high_risk_generator': high_risk_patients()
    }
```

```
}
```

Q5.2 (Reasoning): How does a generator expression maintain state internally between iterations? Why would you use a generator instead of a list comprehension when processing millions of patient records?

6. Python Functions

Positional & Keyword Arguments

Q6.1 (Coding): Design a flexible patient appointment scheduling function:

```
def schedule_appointment(patient_id, doctor_id, *appointment_details,
                           **scheduling_options):
    """
    Create flexible appointment scheduling:
    - Required: patient_id, doctor_id
    - Variable positional: date, time, duration, appointment_type
    - Keyword arguments: priority, notes, insurance_verification, etc.

    Handle different calling patterns:
    schedule_appointment("P123", "D456", "2024-02-15", "10:00")
    schedule_appointment("P123", "D456", priority="urgent", notes="follow-up")
    """
    pass

def update_patient_record(patient_id, /, name=None, age=None, *,
                           blood_type=None, emergency_contact=None):
    """
    Demonstrate positional-only and keyword-only arguments
    - patient_id must be positional
    - blood_type and emergency_contact must be keyword-only
    """
    pass
```

Recursion

Q6.2 (Coding): Implement recursive functions for medical data structures:

```
def calculate_family_risk_score(family_tree: dict, condition: str,
                               current_person: str, generation: int = 0) -> float:
    """
    Recursively calculate genetic risk score based on family history:

    family_tree = {
        'john': {
            'conditions': ['diabetes', 'hypertension'],
            'parents': ['mary', 'robert'],
            'children': ['jane', 'mike']
        },
        'mary': {
            'conditions': ['diabetes'],
            'parents': [],
            'children': ['john']
        }
    }

    Risk scoring:
    - Direct parent/child: 0.5 * base_risk
    - Grandparent/grandchild: 0.25 * base_risk
    - Great-grandparent: 0.125 * base_risk
    """
    pass


def find_medication_interactions(drug: str, interaction_tree: dict,
                                  visited: set = None) -> set:
    """
    Recursively find all possible drug interactions in a complex network
    """
    if visited is None:
        visited = set()

    # Implement recursive interaction detection
    pass
```

Anonymous Functions (Lambda)

Q6.3 (Coding): Use lambda functions for medical data processing:

```
def process_lab_results(lab_data: list) -> dict:
    """
    Use lambda functions with map, filter, and sorted for lab result processing:

    lab_data = [
        {'test': 'glucose', 'value': 120, 'normal_range': (70, 100)},
        {'test': 'cholesterol', 'value': 180, 'normal_range': (0, 200)},
        {'test': 'hemoglobin', 'value': 12.5, 'normal_range': (12, 16)}
    ]
    """

    # Lambda to check if result is abnormal
    abnormal_results = list(filter(# lambda here, lab_data))

    # Lambda to calculate how far from normal range
    severity_scores = list(map(# lambda here, lab_data))

    # Lambda for custom sorting by severity
    sorted_by_priority = sorted(lab_data, key=# lambda here)

    return {
        'abnormal': abnormal_results,
        'severity_scores': severity_scores,
        'priority_order': sorted_by_priority
    }
```

Local and Global Scope

Q6.4 (Coding): Demonstrate scope management in medical system:

```
# Global configuration
HOSPITAL_CONFIG = {
    'max_patients_per_doctor': 50,
    'emergency_threshold_temp': 104.0,
    'default_appointment_duration': 30
}

def patient_management_system():
```

```

"""
Demonstrate proper scope management:
- Global variables for hospital configuration
- Local variables for patient processing
- Proper use of global keyword when needed
"""

def assign_doctor(patient_severity: str):
    # Access global config appropriately
    # Use local variables for calculations
    # Modify global state when necessary (use global keyword)
    pass

def update_hospital_capacity(new_limit: int):
    # Properly modify global configuration
    pass

return assign_doctor, update_hospital_capacity

```

Q6.5 (Reasoning): Explain the difference between local and global scope in Python compared to Java. How does the LEGB rule apply when accessing hospital configuration data from nested functions?

7. Built-in Functions

Q7.1 (Coding): Comprehensive use of built-in functions for medical data:

```

def medical_data_analysis(patient_records: list, lab_results: list) -> dict:
    """
    Use various built-in functions to analyze medical data:

    patient_records = [
        {'id': 'P001', 'age': 45, 'medications': ['aspirin', 'lisinopril']},
        {'id': 'P002', 'age': 67, 'medications': ['metformin', 'aspirin']},
    ]

    lab_results = [
        ('P001', [120, 80, 98.6]), # glucose, bp_sys, temp

```

```

        ('P002', [140, 90, 99.1]),
    ]
    """

    # Use map() to extract patient ages
    ages = list(map(# Your code here))

    # Use filter() to find elderly patients (age > 65)
    elderly = list(filter(# Your code here))

    # Use zip() to combine patient data with lab results
    combined_data = list(zip(# Your code here))

    # Use enumerate() to process patients with index
    indexed_patients = list(enumerate(# Your code here))

    # Use reduce() to find patient with most medications
    from functools import reduce
    most_medications = reduce(# Your code here)

    # Use sorted() with custom key for complex sorting
    sorted_by_risk = sorted(# Your code here)

    # Use min/max for vital statistics
    min_age = min(# Your code here)
    max_temp = max(# Your code here)

    # Use type() and id() for debugging patient data issues
    data_types = {pid: type(data) for pid, data in # Your code here}

    # Use range() for generating appointment time slots
    time_slots = list(range(# Your code here))

    return {
        'ages': ages,
        'elderly_patients': elderly,
        'combined_data': combined_data,
        'indexed_patients': indexed_patients,
        'patient_most_meds': most_medications,
        'risk_sorted': sorted_by_risk,
    }

```

```
'statistics': {'min_age': min_age, 'max_temp': max_temp},  
'data_types': data_types,  
'available_slots': time_slots  
}
```

Q7.2 (Reasoning): Compare Python's `reduce()` function with Java 8's `stream reduce()`. How would you use `reduce()` to aggregate patient statistics across multiple hospital departments?

Reasoning Questions

R1: Why is `patient_info = (42)` an integer while `patient_info = (42,)` is a tuple? How does this affect storing single vital sign measurements?

R2: What makes certain objects hashable and suitable as dictionary keys? Why can't you use `['symptom1', 'symptom2']` as a key but `('symptom1', 'symptom2')` works?

R3: How does Python's string interning work? When you have thousands of patient records with repeated values like "Type 2 Diabetes", how does Python optimize memory usage?

R4: What's the difference between `==` and `is` operators? In a healthcare system, when would you use each to compare patient data?

R5: How does the underlying data structure of Python lists (dynamic arrays) affect performance when inserting patient records at different positions? Compare with tuple's immutable structure.

R6: What's the memory overhead difference between an empty list `[]` and empty tuple `()`? Why does this matter when handling large healthcare datasets?

R7: How does Python's garbage collection work with circular references? Give an example with patient-doctor relationships in a hospital management system.

R8: Explain how generator expressions maintain state between iterations. Why would you choose generators over list comprehensions when processing millions of patient records?

R9: How does Python's name resolution work with the LEGB rule? Provide an example with nested functions in a medical diagnosis system.

R10: What happens internally when you use `patient_data.get('blood_type')` vs `patient_data['blood_type']`? When is each approach appropriate in healthcare applications?

Advanced Integration Questions(Give it a try to get familiar with python syntax):

Q8.1: Create a complete patient management class that demonstrates all learned concepts:

```
class PatientManagementSystem:
    """
    Integrate all Python concepts in a comprehensive healthcare system:
    - Use appropriate data types for different medical data
    - Implement type hints throughout
    - Use comprehensions for data processing
    - Apply functional programming concepts
    - Handle different types of medical operations
    """

    def __init__(self):
        # Initialize with appropriate data structures
        pass

    def add_patient(self, **patient_data):
        # Use **kwargs and type validation
        pass

    def update_vitals(self, patient_id: str, *vitals, **metadata):
        # Demonstrate *args and **kwargs usage
        pass

    def generate_report(self, filter_func=lambda p: True):
        # Use lambda functions and generators
        pass

    def batch_process(self, operations: list):
        # Use map, filter, reduce appropriately
        pass
```